

Diagrama de Classes

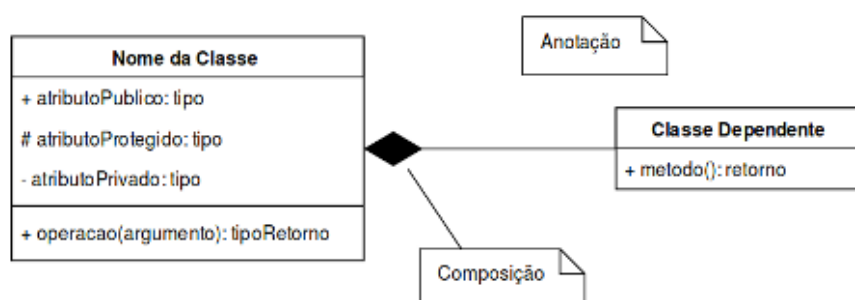
Na programação orientada a objetos (POO), uma classe é um modelo que define as propriedades (atributos) e comportamentos (métodos) que os objetos desse tipo podem ter.

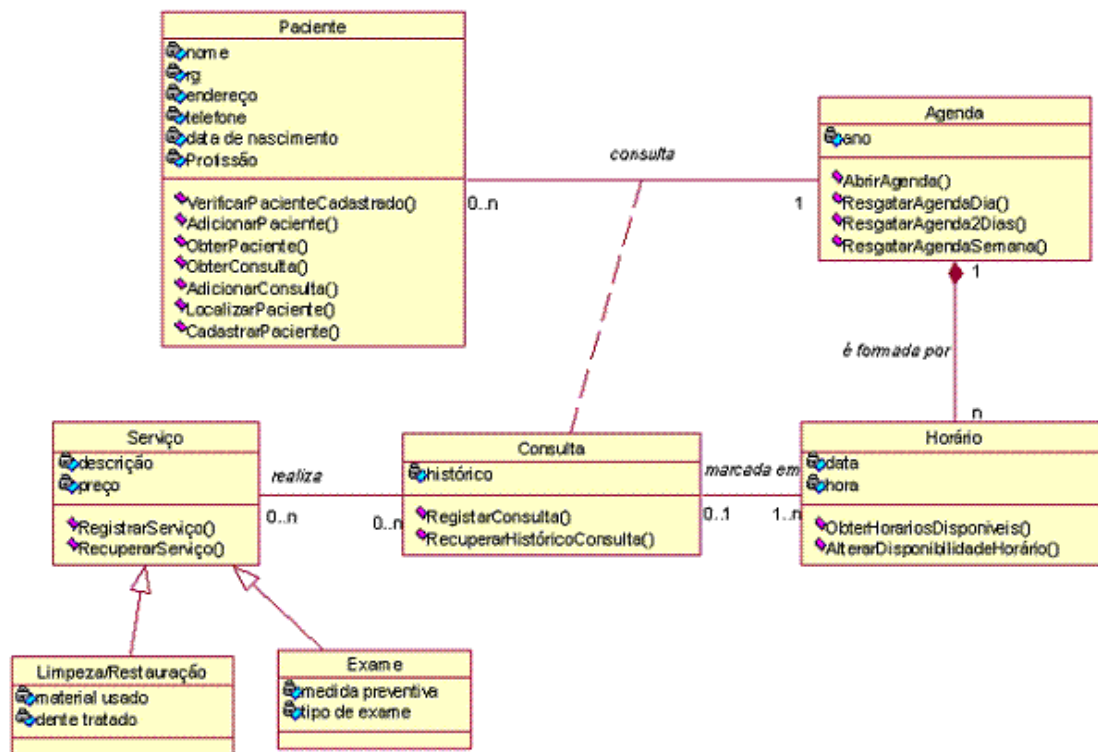
Ele fornece uma visão de alto nível da arquitetura do sistema e das classes que o compõem.

Uma classe é uma representação abstrata de um objeto do mundo real, que encapsula seus atributos e comportamentos.

Um modelo de classes é uma forma de representar essas classes e suas relações de herança, agregação, composição e outros tipos de associações em um diagrama.

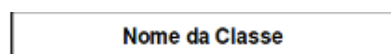
O diagrama de classes é uma ferramenta visual que ajuda a entender a estrutura e o relacionamento entre as classes de um sistema.



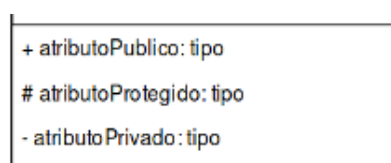


No modelo de classes, você pode ver as classes como retângulos com três compartimentos principais:

1. **Nome da Classe:** O nome da classe é exibido no topo do retângulo e representa o nome da entidade que está sendo modelada. Por exemplo, se estivermos modelando um sistema de gerenciamento de eventos, poderíamos ter uma classe chamada "Evento" para representar os eventos que serão gerenciados.

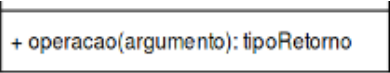


2. **Atributos:** Os atributos são as propriedades ou características que pertencem à classe. Eles são listados no meio do retângulo, logo abaixo do nome da classe. Por exemplo, para a classe "Evento", poderíamos ter atributos como "nome", "data", "local", etc. Esses atributos representam as informações específicas de cada evento.



Modificadores de acesso:

1. **+ (Símbolo de Mais):** Representa a visibilidade pública. Isso significa que o atributo ou método pode ser acessado de qualquer lugar dentro do sistema.
 2. **(Símbolo de Menos):** Representa a visibilidade privada. Elementos com visibilidade privada só podem ser acessados dentro da própria classe em que estão definidos.
 3. **# (Símbolo de Sustenido/Hash):** Representa a visibilidade protegida. Elementos com visibilidade protegida podem ser acessados dentro da própria classe e também por suas subclasses.
-
3. **Métodos:** Os métodos são as ações ou comportamentos que a classe pode executar. Eles são listados na parte inferior do retângulo, logo abaixo dos atributos. Por exemplo, para a classe "Evento", poderíamos ter métodos como "adicionarParticipante", "removerParticipante", "alterarData", etc. Esses métodos representam as operações que podem ser realizadas em um evento.



```
+ operacao(argumento): tipoRetorno
```

O diagrama mostra um retângulo representando uma classe. Na parte inferior do retângulo, há uma linha de texto que indica a assinatura de um método público: "+ operacao(argumento): tipoRetorno". O símbolo "+" indica visibilidade pública.

Modificadores de acesso:

1. **+ (Símbolo de Mais):** Representa a visibilidade pública. Isso significa que o atributo ou método pode ser acessado de qualquer lugar dentro do sistema.
2. **(Símbolo de Menos):** Representa a visibilidade privada. Elementos com visibilidade privada só podem ser acessados dentro da própria classe em que estão definidos.
3. **# (Símbolo de Sustenido/Hash):** Representa a visibilidade protegida. Elementos com visibilidade protegida podem ser acessados dentro da própria classe e também por suas subclasses.

RELACIONAMENTOS

Classes possuem relacionamentos entre elas (para comunicação)

Associação

A associação é um relacionamento geral que indica que uma classe está associada a outra classe de alguma forma.

Ela pode representar diferentes níveis de conexão e interação entre classes. Pode ser bidirecional ou unidirecional, dependendo da necessidade.



```
import java.util.ArrayList;
import java.util.List;

class Estudante {
    private String nome;
    private int idade;
    private List<Curso> cursosMatriculados;

    public Estudante(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
        this.cursosMatriculados = new ArrayList<>();
    }

    public void matricularEmCurso(Curso curso) {
        cursosMatriculados.add(curso);
        curso.adicionarEstudante(this);
    }

    // Getters e setters

    // Outros métodos relevantes
}
```

```

}

class Curso {
    private String nome;
    private int credits;
    private List<Estudante> estudantesMatriculados;

    public Curso(String nome, int credits) {
        this.nome = nome;
        this.credits = credits;
        this.estudantesMatriculados = new ArrayList<>();
    }

    public void adicionarEstudante(Estudante estudante) {
        estudantesMatriculados.add(estudante);
    }

    // Getters e setters

    // Outros métodos relevantes
}

```

Multiplicidade

A multiplicidade no diagrama de classes da UML (Unified Modeling Language) é uma maneira de especificar quantos objetos podem participar de um determinado relacionamento entre classes.

Ela define a cardinalidade ou o número de instâncias de uma classe que podem estar associadas a outra classe por meio de uma relação.

- **Números Exatos:**

- **0..1**: Zero ou um objeto.
- **1**: Exatamente um objeto.
- **0..*** ou : Qualquer número de objetos, incluindo zero.
- **1..***: Pelo menos um objeto.

- **Intervalos de Números:**

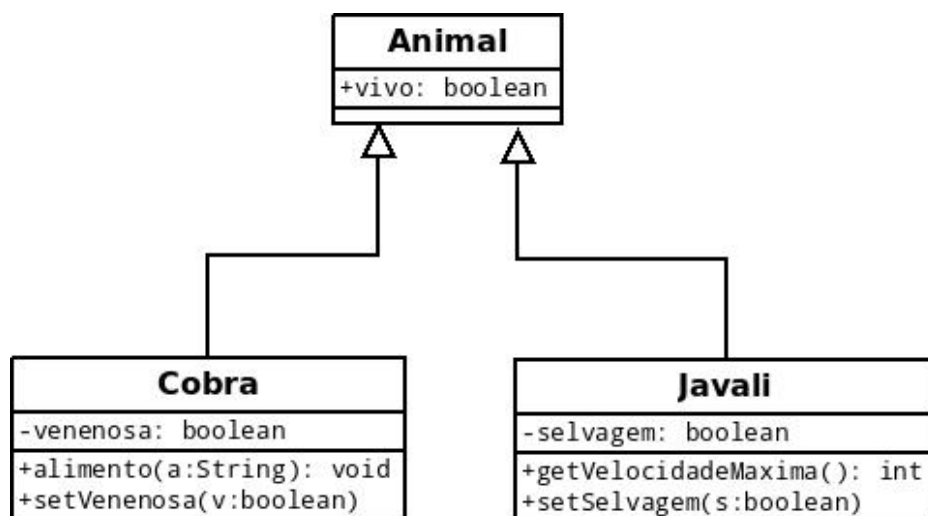
- **x..y** : Entre x e y objetos, incluindo ambos os extremos.

Exemplo de uso de multiplicidade em uma associação entre "Estudante" e "Curso":

- Estudante: **1..*** (um ou mais estudantes podem estar associados a um curso)
- Curso: **0..*** (um curso pode ter zero ou mais estudantes matriculados)

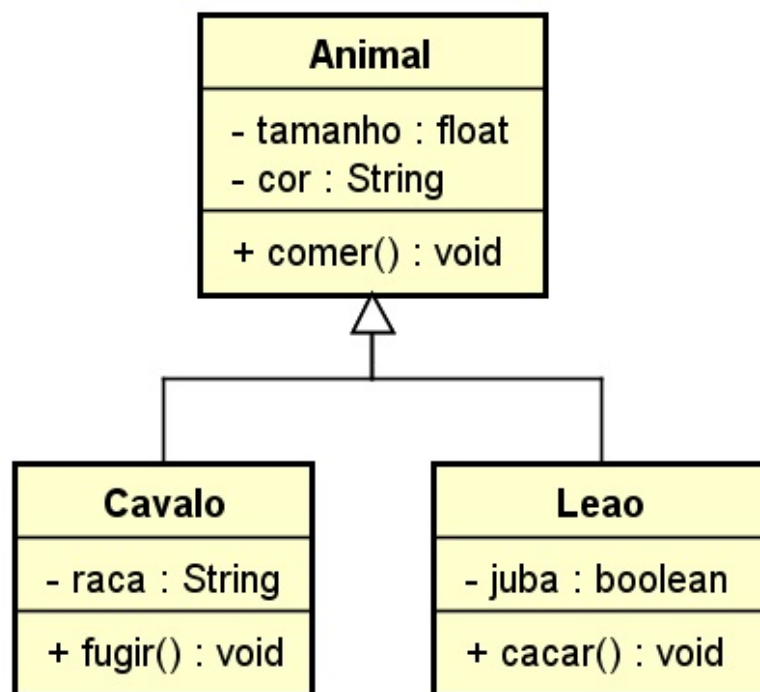
Herança ou Generalização

A herança permite que uma classe herde os atributos e métodos de outra classe, estabelecendo uma hierarquia entre as classes.



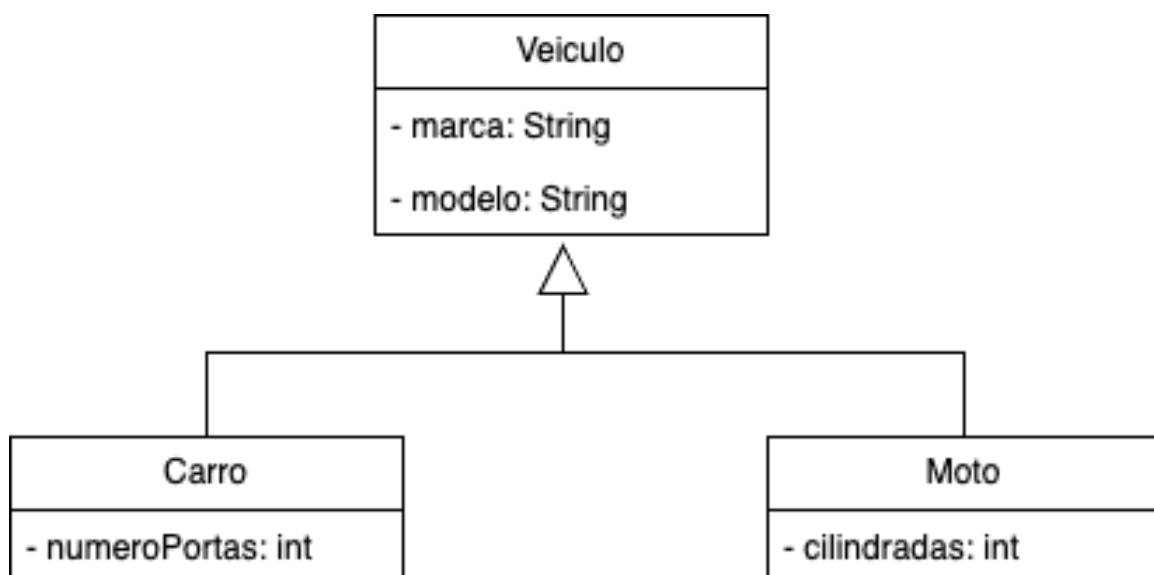
No diagrama de classes da UML, a relação de herança é representada por uma linha sólida com uma seta apontando da classe derivada (subclasse) para a classe base (superclasse).

A seta indica a direção da herança, ou seja, a classe derivada herda da classe base.



Você pode ter uma classe "Animal" como superclasse, com classes derivadas como "Cavalo", "Leao".

Cada classe filho herdando os atributos e métodos básicos da classe pai "Animal" e adicionando seus próprios atributos e comportamentos específicos.



```
class Veiculo {
    private String marca;
```

```

private String modelo;

public Veiculo(String marca, String modelo) {
    this.marca = marca;
    this.modelo = modelo;
}

// Getters e setters para marca e modelo
}

class Carro extends Veiculo {
    private int numeroPortas;

    public Carro(String marca, String modelo, int numeroPortas) {
        super(marca, modelo);
        this.numeroPortas = numeroPortas;
    }

    // Getter e setter para número de portas
}

class Moto extends Veiculo {
    private int cilindradas;

    public Moto(String marca, String modelo, int cilindradas) {
        super(marca, modelo);
        this.cilindradas = cilindradas;
    }

    // Getter e setter para cilindradas
}

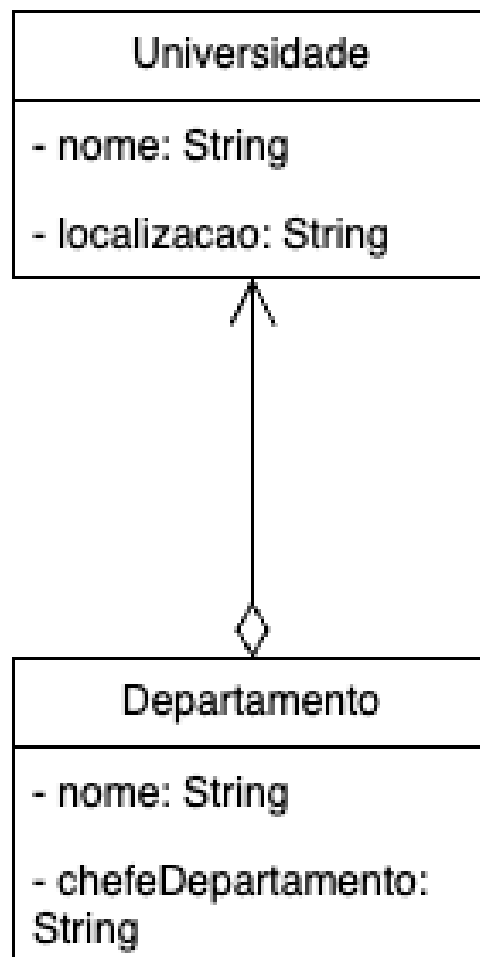
```

Agregação - Foco na implementação

A agregação é uma forma de associação que indica que uma classe "todo" é composta por outras classes "parte".

As partes podem existir independentemente da classe "todo" e podem ser compartilhadas por várias instâncias.

O relacionamento é representado por um losango sem preenchimento.



```
import java.util.ArrayList;
import java.util.List;

class Universidade {
    private String nome;
    private String localizacao;
    private List<Departamento> departamentos;

    public Universidade(String nome, String localizacao) {
        this.nome = nome;
    }
}
```

```

        this.localizacao = localizacao;
        this.departamentos = new ArrayList<>();
    }

    public void adicionarDepartamento(Departamento departam
ento) {
        departamentos.add(departamento);
    }

    // Getters e setters
}

class Departamento {
    private String nome;
    private String chefeDepartamento;

    public Departamento(String nome, String chefeDepartamen
to) {
        this.nome = nome;
        this.chefeDepartamento = chefeDepartamento;
    }

    // Getters e setters
}

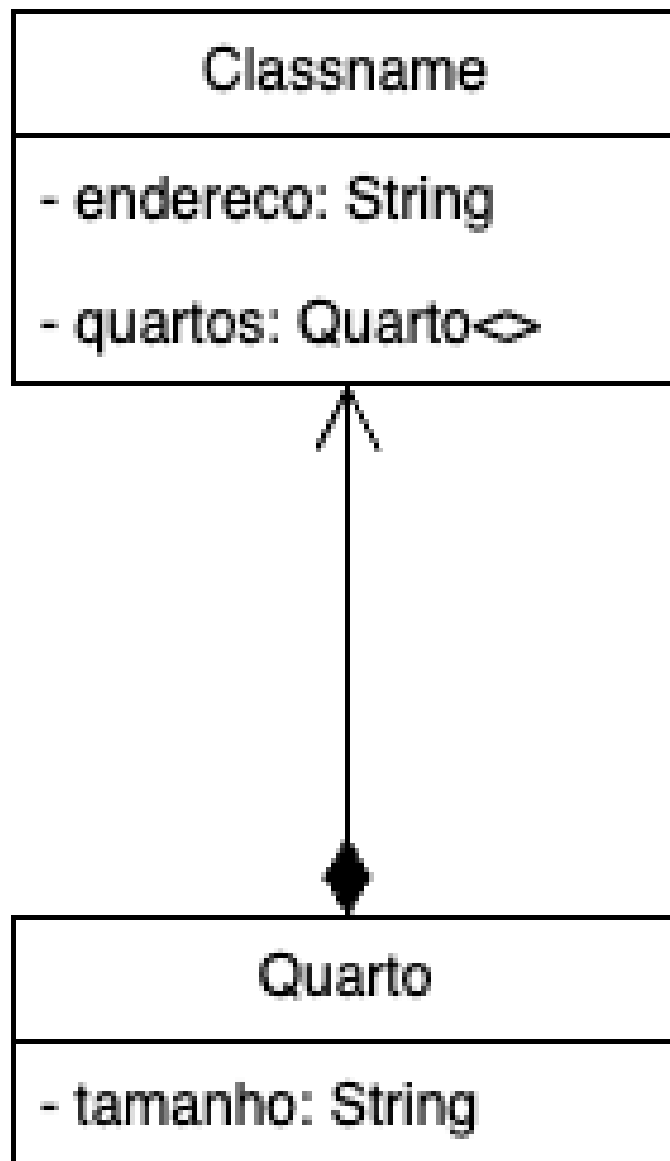
```

Composição - Foco na implementação

A composição é semelhante à agregação.

Indica que as partes não podem existir sem a classe "todo" e, se a classe "todo" for excluída, as partes também serão.

O relacionamento é representado por um losango preenchido.



```
import java.util.ArrayList;
import java.util.List;

class Casa {
    private String endereco;
    private List<Quarto> quartos;

    public Casa(String endereco) {
        this.endereco = endereco;
        this.quartos = new ArrayList<>();
    }
}
```

```
public void adicionarQuarto(Quarto quarto) {
    quartos.add(quarto);
}

// Getters e setters
}

class Quarto {
    private String tamanho;

    public Quarto(String tamanho) {
        this.tamanho = tamanho;
    }

    // Getters e setters
}
```

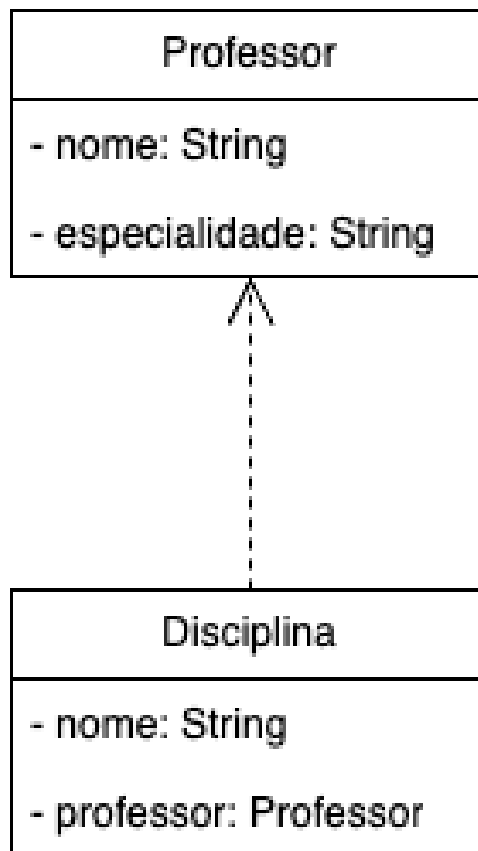
Dependência

A dependência indica que uma classe depende de outra de alguma forma

Pode ocorrer quando uma classe usa os serviços ou colabora com outra classe.

Não há uma associação permanente ou estrutural entre elas.

Uma maneira de expressar que uma classe usa os serviços ou colabora com outra classe



```
class Professor {
    private String nome;
    private String especialidade;

    public Professor(String nome, String especialidade) {
        this.nome = nome;
        this.especialidade = especialidade;
    }

    // Getters e setters
}

class Disciplina {
    private String nome;
    private Professor professor;

    public Disciplina(String nome, Professor professor) {
        this.nome = nome;
        this.professor = professor;
    }
}
```

```
}  
  
// Getter e setter para professor  
}
```